

Webキャッシュ戦略の 変遷を理解する

SPA → SSR → ISR/Edge

TECH LT・2025.01

アジェンダ

1. SPA時代 - クライアントサイドレンダリングとimmutableアセット戦略
2. SSR/Jamstack時代 - サーバーサイドレンダリング回帰と静的サイト生成
3. ISR/Edge時代 - stale-while-revalidateとEdge Computeの活用
4. API単位のキャッシュ最適化 - データソースごとの最適化でオリジン負荷削減
5. まとめ - 各時代の比較と今後の展望

SPA時代

背景

React（2013）、Vue（2014）の登場により、クライアントサイドでUIを構築するSPAが主流に

主な技術スタック

React

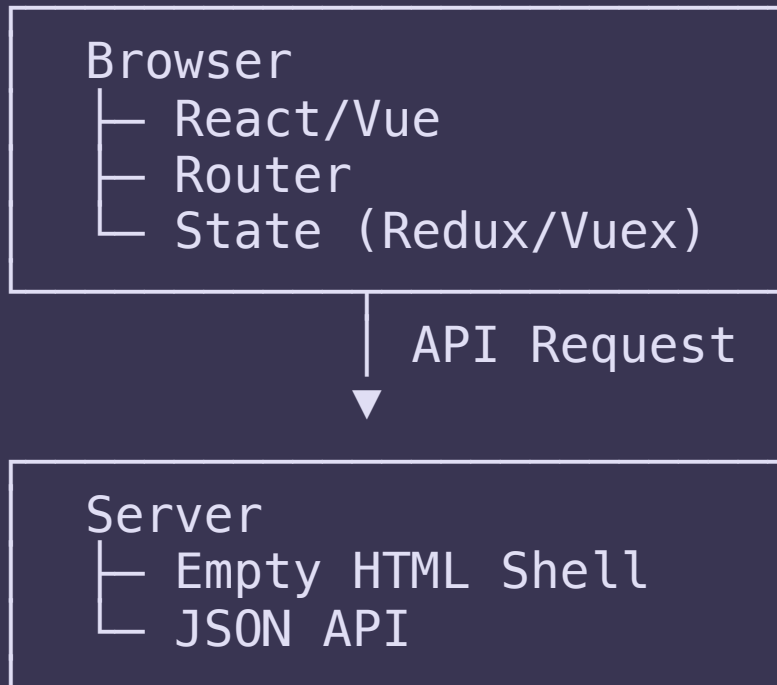
Vue.js

Angular

webpack

REST API

アーキテクチャ



SPA時代のキャッシュ戦略

index.html

空のシェル - キャッシュ短め

```
Cache-Control: no-cache
```

app.[hash].js

content hash付き - 永続キャッシュ

```
Cache-Control:  
max-age=31536000, immutable
```

⚠ 初回表示の遅さ

JSバンドルのダウンロード・パース・実行が完了するまでコンテンツが表示されない

⚠ SEO問題

クローラーがJSを実行できないとコンテンツを認識できない。OGP取得も困難

レンダリングフロー

1. 空のHTML受信
- ↓
2. JS/CSSダウンロード
- ↓
3. JSパース・実行
- ↓
4. APIデータ取得
- ↓
5. コンテンツ表示 ✓

 ユーザーはここまで待つ必要あり

SSR/Jamstack時代

SPAの「初回表示の遅さ」「SEO問題」を解決するため、サーバーでHTML生成する方式が再注目

SSR - リクエスト毎にサーバーでレンダリング

Next.js / Nuxt.js

SSG (Jamstack) - ビルド時に全ページを静的生成

Gatsby / Hugo

処理の流れ

SSR フロー

Request → [Server Render] → HTML

SSG フロー

[Build] → Static HTML → [CDN] → User

✓ 解決されたこと

初回表示でコンテンツが含まれたHTMLが返る

→ 高速表示 & SEO対応

SSR/Jamstack時代のキャッシュ戦略

SSRのキャッシュ課題

問題点: リクエスト毎にレンダリング = 高負荷

対策:

- ページ単位での短いTTL設定
- パーソナライズ部分のみCSR

Jamstackの戦略

シンプルな解決策: 全ページをビルド時に生成してCDNに配置

デプロイフロー:

```
git push → Build → CDN Deploy → Purge
```

100%キャッシュヒット可能！

CDNキャッシュの活用が本格化

User → CDN Edge (Cache Hit!) → Origin

SSR/Jamstack時代の課題

SSRの課題

-  **サーバー負荷** - リクエスト毎にレンダリング処理
-  **TTFB悪化** - レンダリング完了まで待機
-  **コスト** - 常時稼働のサーバーが必要

Jamstackの課題

-  **ビルド時間** - ページ数に比例して増加
-  **更新頻度の制限** - 頻繁な更新には全ビルド
-  **動的コンテンツ** - パーソナライズに弱い

ECサイトでの問題例

商品数 **10万点** のサイトで、1商品変更のために全再ビルドは非現実的

これらの課題を解決するために... → ISR/Edgeの登場

ISR/Edge時代（2020～現在）

ISR（Incremental Static Regeneration）

静的生成の高速さとSSRの鮮度を両立（Next.js 9.5～）

stale-while-revalidate パターン

```
Cache-Control:  
max-age=1, stale-while-revalidate=59
```

max-age=1: 1秒間は「新鮮」として即座にキャッシュから返却

stale-while-revalidate=59: 1秒経過後～60秒間は「古いが使える」期間

→ 古いキャッシュを即座に返しつつ、バックグラウンドで再生成

Edge Computeの成熟

Cloudflare Workers

Vercel Edge

CloudFront Functions

ISR の動作フロー

1. 初回アクセス

オンデマンドで生成 → キャッシュ保存

2. 2回目以降

キャッシュから即座にレスポンス

3. **revalidate** 時間経過後

古いキャッシュを返却（ユーザーは待たない）

4. バックグラウンド再生成

次回アクセス時は新しいコンテンツ

 ユーザーは常に即座にレスポンスを受け取る

SEOとWeb Core Vitalsの変化

Googleの評価基準の進化（2020～）

- **LCP (Largest Contentful Paint)** - メインコンテンツの表示速度
- **FID/INP (First Input Delay / Interaction to Next Paint)** - インタラクティブ性
- **CLS (Cumulative Layout Shift)** - レイアウトの安定性

ISRが有利な理由

- ✓ サーバーレンダリングHTML → LCP改善
初回からコンテンツ含むHTMLを返却
- ✓ エッジキャッシュ → TTFB高速化
オリジンまで行かずに即座にレスポンス
- ✓ ハイドレーション最適化 → FID/INP改善
段階的なJSロードで早期インタラクション可能

Core Web Vitalsがランキング要素に

2021年6月より、GoogleがCore Web Vitalsを検索ランキングのシグナルとして採用

ISR採用による直接的なメリット:

- 検索順位向上
- オーガニックトラフィックの増加
- ビジネス成果への貢献

 **技術選択がSEO戦略に直結する時代へ**

現代のキャッシュ戦略

多層キャッシュアーキテクチャ

Browser (Service Worker) → Edge / CDN (KV Store / ISR) → Origin (Redis / DB)

コンテンツ別キャッシュ戦略

コンテンツ	戦略	TTL例
静的アセット	immutable	1年
商品一覧	ISR	60秒
商品詳細	SWR	1秒 + 59秒
カート・決済	no-store	キャッシュなし

Edgeでできること

- ✓ キャッシュキー制御
- ✓ A/Bテスト
- ✓ 認証チェック
- ✓ 地域別コンテンツ
- ✓ Bot検出

API単位のカッシュ最適化

ページ単位 → API単位へ

従来はページ全体で統一されたTTL。現代はデータソースごとに最適化

実装例 (Next.js)

```
async function ProductPage({ id }) {  
  const [product, stock] =  
    await Promise.all([  
      // 商品情報: 1時間  
      fetch(`/api/products/${id}`, {  
        next: { revalidate: 3600 }  
      }),  
      // 在庫: 10秒  
      fetch(`/api/stock/${id}`, {  
        next: { revalidate: 10 }  
      })  
    ])  
  return <div>...</div>  
}
```


メリット

1. オリジン負荷削減

変更のないAPIはキャッシュヒット

2. レスポンスタイム最適化

並列取得 + 個別TTLで最速構成

3. データベース負荷分散

更新頻度に応じたアクセス制御

データ特性別の戦略

データ	TTL	理由
商品マスタ	1時間	変更少
在庫数	10秒	リアルタイム性
レビュー	5分	適度な鮮度
カート	なし	即時反映

時代の比較

	SPA	SSR/SSG	ISR/Edge
キャッシュ対象	JS/CSSアセット	静的HTML全体	動的HTML + Edge処理
初回表示速度	△ 遅い	○ 普通～速い	◎ 常に高速
更新反映	◎ 即時（API経由）	△ 再ビルド必要	◎ 自動再生成
SEO	△ 要対策	◎ 良好	◎ 良好
オリジン負荷	○ API負荷あり	○ SSRは高負荷	◎ Edgeで分散
実装複雑さ	○ シンプル	○ やや複雑	△ 設計が重要

💡 ISRは「過去の手法の良いところ取り」をフレームワークが抽象化したもの

まとめ

キャッシュ戦略の進化

「静的 vs 動的」の二択から、ISR/SWRで両立可能に。設計次第でUXとコストを大幅改善

Edgeの活用

単なるキャッシュから「処理もできるエッジ」へ。Core Web Vitals、負荷軽減、コスト削減の鍵

おしまい